

2024-2학기

Cloud Computing Term Project 2: Serverless Implementation

학번: 2019312014

이름: 박병준

학과: 컴퓨터교육과

과목: 클라우드컴퓨팅개론

담당 교수: 최윤석 교수님

목차

1. 프로젝트 소개
2. 서버리스로 구현한 부분 설명
3. 서버리스 컴퓨팅의 장점
4. 서버리스 선택 시 고려해야 할 요소
5. 서버리스 구조의 단점 및 보완 방안
6. 결과 요약

1. 프로젝트 소개

프로젝트의 목적은 Term Project 1의 일부 기능을 AWS Lambda를 사용해 Serverless로 구현하는 것이다. 이를 위해 Term Project 1의 공방 소개 웹사이트를 기반으로 프로젝트를 수행했으며, AWS Lambda와 AWS API Gateway를 연결해 HTTP request를 통해 Lambda를 호출하였다. 이렇게 리팩터링된 웹페이지는 AWS Lambda 콘솔의 모니터링을 사용하거나 로그를 확인해 실행된 Lambda 함수가 정상적으로 작동하는 지 판단할 수 있다.

2. 서버리스로 구현한 부분 설명

Term Project 1의 공방 소개 웹사이트 기능들 중 "공BTI 테스트" 부분을 서버리스로 구

현했다. 공방 소개 웹사이트의 구조는 Term Project 1과 동일하게 11개의 html 파일, 12개의 css 파일, 3개의 js 파일, 12개의 image 파일로 구성되어 있으며, 기능적 수정사항은 없다. 소스코드는 다음 github 주소에 배포되어 있다.

https://github.com/bjpark0925/young_craft_workshop_website

웹사이트는 다음 주소를 통해 접근할 수 있다.

https://bjpark0925.github.io/young_craft_workshop_website/main.html

공BTI 테스트 부분은 공예 초심자에 속하는 대부분의 웹사이트 사용자를 위해 자신의 성향에 맞는 공예 프로그램을 추천해주는 기능을 주로 제공한다. 마치 MBTI 테스트를 하는 것처럼 주어진 문장을 보고 답을 할 수 있으며, 선택한 답을 기반으로 프로그램에서 결과를 제공하는 것이 특징이다. 결과는 '전략을 잘 세우는 당신은 포크아트가 어울립니다. (생활 집기에 고풍스러운 그림 그리기)'처럼 간단한 텍스트로 표현된다.

이 공BTI 테스트 부분을 서버리스로 구현하기 위해, 공BTI 테스트의 결과 계산 로직을 Lambda 함수로 구현하여 Lambda는 이 결과를 계산하고 API Gateway를 통해 HTTP 요청으로 반환하도록 했다. 구체적으로, gbt.js의 answerQuestion 함수와 showResult 함수에서 작성된 코드의 로직을 Lambda에서 처리하도록 코드의 구조를 변경했다. 이를 통해, 사용자가 질문에 답변한 데이터를 Lambda로 전달하면, Lambda가 코드 내에서 정의한 flag 배열을 기반으로 결과를 계산하여 반환하게 된다. 코드의 구조를 변경했지만, 기능의 수행은 전과 동일하게 이루어져, 서버리스 환경으로의 migration이 성공적으로 이루어졌음을 확인했다.

다음은 실제로 프로젝트를 수행하며 캡처한 화면들이다.

-새 Lambda 함수 생성

The screenshot shows the AWS Lambda console interface for creating a new function. The 'New function' tab is selected, and the 'Function name' is set to 'my-serverless-function'. The 'Runtime' is set to 'Python 3.13' and the 'Architecture' is 'x86_64'. The 'Permissions' section shows 'Basic execution role'. On the right, there's a 'Tutorials' section with a link to 'Getting started'.

-AWS API Gateway 생성 및 연결

The screenshot shows the AWS API Gateway console for the API named 'GBTITestAPI'. The left sidebar contains navigation links for API Gateway, API, 리소스, and various API settings. The main area displays the resource configuration for the path '/gbti-result' with the HTTP method 'POST'. The resource ID is 'v470bv'. A flow diagram illustrates the request flow: 클라이언트 (Client) sends a request to 메서드 요청 (Method Request), which is then processed by 통합 요청 (Integration Request) and sent to the Lambda function. The response flows back through 통합 응답 (Integration Response) and 메서드 응답 (Method Response) to the 클라이언트. The '메서드 요청 설정' (Method Request Settings) section shows '권한 부여' (Authorization) as NONE and '요청 검사기' (Request Validator) as None. The 'API 키가 필요함' (API Key Required) checkbox is unchecked, and the 'SDK 작업 이름' (SDK Operation Name) is set to '메서드 및 경로에 따라 생성됨' (Generated based on method and path).

-gbt.js 코드 수정

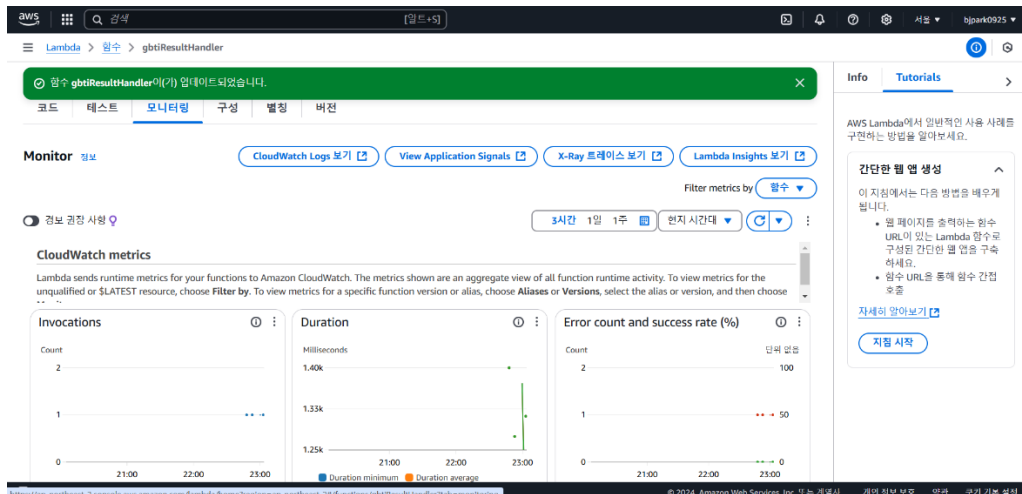
The screenshot shows a code editor with the file 'gbt.js' open. The code is written in JavaScript and includes a function 'showQuestion' that fetches data from an API and updates the DOM. The code is as follows:

```
function showQuestion(answer) {
  currentQuestion++;
  showQuestion();
}

async function showResult() {
  const response = await fetch('https://ub9xd44.execute-api-northeast-2.amazonaws.com', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ flag: flag })
  });
  const data = await response.json();
  document.getElementById('result-text').textContent = data.result;
  document.getElementById('question-page').classList.add('hidden');
  document.getElementById('result-page').classList.remove('hidden');
}

function restartTest() {
  currentQuestion = 0;
  flag = [false, false, false];
  document.getElementById('result-page').classList.add('hidden');
}
```

-AWS Lambda 모니터링



-로그 데이터

CloudWatch Logs

Lambda logs all requests handled by your function and automatically stores logs generated by your code through Amazon CloudWatch Logs. To validate your code, instrument it with custom logging statements. The following tables list the most recent and most expensive function invocations across all function activity. To view logs for a specific function

Recent invocations

#	Timestamp	RequestId	LogStream	DurationInMS	BilledDurationInMS
1	2024-12-19T23:03:37.892+09:00	c605f17c-236b-4ed5-a098-5d9e495a8665	2024/12/19/[SLATEST]3732b2ffc9ea47bc9988c4fc579c78ff	1313.78	1314.
2	2024-12-19T23:01:24.795+09:00	ee3084ff-013f-4404-915d-2528d2e3047a	2024/12/19/[SLATEST]2d3b9821ba214c2eb5a0051303d258bf	1254.15	1255.
3	2024-12-19T23:00:57.056+09:00	e73bb608-2cf2-4ed6-a069-d1ec67be084a	2024/12/19/[SLATEST]2d3b9821ba214c2eb5a0051303d258bf	1371.68	1372.
4	2024-12-19T22:53:03.776+09:00	7999583c-2530-4077-9481-d9d24bd78ac4	2024/12/19/[SLATEST]a8f8ccdfc494a3b9dc06095828c782	1278.01	1279.
5	2024-12-19T22:48:56.153+09:00	f7b30121-0fdf-4a3c-88d5-a086ca5e697d	2024/12/19/[SLATEST]a8f8ccdfc494a3b9dc06095828c782	1399.69	1400.

Most expensive invocations in GB-seconds (memory assigned * billed duration)

#	Timestamp	RequestId	LogStream	BilledDurationInMS
1	2024-12-19T22:48:56.153+09:00	f7b30121-0fdf-4a3c-88d5-a086ca5e697d	2024/12/19/[SLATEST]a8f8ccdfc494a3b9dc06095828c782	1400.0
2	2024-12-19T23:00:57.056+09:00	e73bb608-2cf2-4ed6-a069-d1ec67be084a	2024/12/19/[SLATEST]2d3b9821ba214c2eb5a0051303d258bf	1372.0
3	2024-12-19T23:03:37.892+09:00	c605f17c-236b-4ed5-a098-5d9e495a8665	2024/12/19/[SLATEST]3732b2ffc9ea47bc9988c4fc579c78ff	1314.0
4	2024-12-19T22:53:03.776+09:00	7999583c-2530-4077-9481-d9d24bd78ac4	2024/12/19/[SLATEST]a8f8ccdfc494a3b9dc06095828c782	1279.0

간단한 웹 앱 생성

이 지침에서는 다음 방법을 배우게 됩니다.

- 웹 페이지를 출력하는 함수 URL이 있는 Lambda 함수로 구성된 간단한 웹 앱을 구축하세요.
- 함수 URL을 통해 함수 간접 호출

자세히 알아보기

지침 시작

3. 서버리스 컴퓨팅의 장점

서버리스 컴퓨팅은 개발자가 서버 관리를 직접 하지 않고, 클라우드 제공자가 자원을 자동으로 할당하여 애플리케이션 실행을 처리하는 방식이다. 장점으로는 첫째로, 애플리케이션에 대한 요청이 증가하면 사용자가 신경 쓰지 않아도 클라우드 제공자가 필요한 만큼의 리소스를 할당해주므로 확장성 측면에서 용이하다. 둘째로, 서버리스는 사용한 만큼만 비용을 지불하므로 서버를 항상 실행할 필요가 없고, 요청이 있을 때만 자원이 사용되므로 고정 비용이 적다. 따라서 트래픽이 일정하지 않거나, 요청이 간헐적으로 발생하는 애플리케이션에 적합하며 비용을 최적화할 수 있다. 셋째로, 코드를 배포하는 과정이 간단하고 빠르다. 코드의 변경 사항이 바로 반영되며, 서버 설정이 필요하지 않다. 마지막으로, 자동 장애 복구 및 관리를 클라우드 제공자가 담당하므로 장애에 대해 걱정하지 않아도 된다.

4. 서버리스 선택 시 고려해야 할 요소

서버리스를 개발 환경으로 선택하기 전에 고려해야 할 요소로는 우선, 서버리스를 개발 환경으로 채택했을 때의 기회비용을 충분히 고려해야 한다. 예를 들면 온프레미스 환경을 비롯해 AWS EC2, AWS Lambda 등을 모두 비교해 특정 환경별로 서비스를 운영할 때 생길 장단기적 비용을 잘 따져봐야 한다. 복잡도가 낮은 코드를 사용하거나 트래픽이 드물게 발생하는 서비스를 단기적으로 운영할 경우 서버리스 환경이 적합하다.

서버리스를 개발 환경으로 선택했다면, 프로그램 전체를 서버리스 환경으로 설정할 것인지, 특정 서비스만 서버리스 환경으로 설정할 것인지 고려해야 한다. 특정 서비스만 서버리스 환경으로 설정한다면 각 서비스가 독립적으로 운영되므로, 서비스 간 통신을 관리하는 데 복잡성이 증가할 수 있음에 유의해야 한다. 이때 REST API(웹 기반 시스템에서

서로 다른 소프트웨어 컴포넌트 간의 통신을 가능하게 해주는 인터페이스)와 같은 통신 방법을 사용할 수 있다. 그리고 서버리스 환경에서 개별 서비스의 장애가 전체 시스템에 영향을 주지 않도록 장애 복구 전략을 세워야 한다. 자동 확장과 장애 복구 기능이 있는 서버리스 플랫폼의 장점을 최대한 활용하는 것이 중요하다. 마지막으로, 기능별로 보안 정책을 다르게 적용함으로써, 필요한 부분에만 보안 강화를 집중할 수 있어야 한다. AWS Cognito 같은 인증 서비스로 OAuth(ex. 카카오톡 인증하기) 등을 지원하거나, API Gateway를 통해 인증된 사용자가 접근할 수 있도록 하는 식으로 보안 문제를 해결할 수 있어야 한다.

5. 서버리스 구조의 단점 및 보완 방안

서버리스 구조의 단점으로는 첫째로, 콜드 스타트 레이턴시가 발생할 수 있다. 콜드 스타트란 서버리스 환경에서 함수가 호출될 때 클라우드 제공자가 처음으로 자원을 할당하는데, 이 과정에서 지연이 발생하는 경우를 말한다. 둘째로, 비용 예측이 어렵다. 서버리스 아키텍처는 사용량에 따라 비용이 발생하므로, 사용 패턴에 따라 예측하지 못한 비용이 발생할 수 있다. 특히 트래픽이 급증하면 예상보다 높은 비용을 지불해야 할 수 있다. 셋째로, 성능 튜닝이나 서버 설정에 대한 세밀한 제어가 어렵다. 따라서 이를 보완하기 위해서는 서비스나 기능별로 서버리스 환경 이외에도 AWS EC2 환경, 온프레미스 환경 등을 적절히 나누어 활용해볼 수 있다. 이렇게 하면 성능 튜닝이나 서버 설정의 자율성에 대한 결핍을 해소할 수 있고, 서버리스 대신 다른 환경을 채택하여 사용 패턴이 불규칙해 발생하는 비용 변동성 문제를 해결할 수 있다.

6. 결과 요약

지금까지 공방 소개 웹사이트의 일부 기능을 AWS Lambda를 사용해 Serverless로 구현하고, AWS API Gateway를 연결해 HTTP request를 통해 Lambda를 호출하였으며, AWS Lambda 콘솔의 모니터링을 사용해 정상 작동 여부를 평가할 수 있었다. 이러한 실습을 통해 서버리스 환경에 대한 이해가 더 깊어질 수 있었다. 서버리스 컴퓨팅은 클라우드 제공자가 서버 관리를 맡아 개발자가 확장성, 비용, 배포 속도에서 이점을 얻을 수 있는 방식이다. 트래픽이 일정하지 않은 애플리케이션에서 사용량 기반 비용 지불로 비용 최적화가 가능하며, 장애 복구도 자동으로 이루어진다. 그러나 선택 시에는 온프레미스, EC2 등과 비교해 장단기적 비용과 복잡성을 고려해야 하며, 서버리스 환경 내에서도 서비스 간 통신과 장애 복구 전략을 설계해야 한다. 단점으로는 콜드 스타트로 인한 지연, 예측하기 어려운 비용, 성능 튜닝 제약이 있으며, 이를 보완하기 위해 서버리스와 다른 환경을 병행 활용하는 하이브리드 아키텍처를 사용하는 것이 효과적이다.